Bootstrap 부트스트랩

개요

Bootstrap 모듈은 IPFS 노드가 네트워크에 연결하고 충분한 피어 연결을 유지하는 데 필요한 부트스트랩 기능을 제공합니다. 이 모듈은 주기적으로 연결된 피어 수를 확인하고, 필요시 잘 알려진 부트스트랩 피어들에게 연결을 시도합니다. 또한 백업 부트스트랩 피어 시스템을 통해 네트워크 안정성을 향상시킵니다.

주요 개념

부트스트랩 프로세스

부트스트랩은 IPFS 노드가 네트워크에 참여하기 위한 초기 연결 과정입니다:

1. **연결 상태 확인**: 현재 연결된 피어 수가 최소 임계값 이상인지 확인
2. **부트스트랩 피어 연결**: 필요시 설정된 부트스트랩 피어들에게 연결 시도
3. **백업 피어 활용**: 기본 부트스트랩 피어 연결 실패 시 백업 피어 사용
4. **라우팅 부트스트랩**: 라우팅 시스템의 자체 부트스트랩 프로세스 시작

백업 부트스트랩 시스템

- **자동 백업**: 연결된 피어들을 주기적으로 백업 부트스트랩 피어로 저장
- **필터링**: 릴레이 전용 피어는 제외하고 직접 주소를 가진 피어만 저장
- **중복 제거**: 기존 부트스트랩 피어와 중복되지 않는 피어만 저장

기본 사용법

기본 부트스트랩 설정

```go
package main

import (
"context"
"log"
"time"
"github.com/ipfs/boxo/bootstrap"
"github.com/libp2p/go-libp2p"
"github.com/libp2p/go-libp2p/core/peer"
"github.com/multiformats/go-multiaddr"
)

func main() {
// libp2p 호스트 생성
host, err := libp2p.New()
if err != nil {
log.Fatal(err)
}
defer host.Close()
// 부트스트랩 피어 정의
bootstrapPeers := []peer.AddrInfo{
```

```
{
ID: "12D3KooWGRPLCuizAiJWLtktrdxKKoKG5b3BtG4pEKbpzTLxmdSZ",
Addrs: []multiaddr.Multiaddr{
multiaddr.StringCast("/ip4/104.131.131.82/tcp/4001"),
},
},
// 더 많은 부트스트랩 피어들...
}
// 부트스트랩 설정 생성
cfg := bootstrap.BootstrapConfigWithPeers(bootstrapPeers)
// 부트스트랩 시작
bootstrapper, err := bootstrap.Bootstrap(host.ID(), host, nil, cfg)
if err != nil {
log.Fatal(err)
}
defer bootstrapper.Close()
// 애플리케이션 로직...
time.Sleep(30 * time.Second)
}
```

사용자 정의 설정으로 부트스트랩

```
func customBootstrap() error {
```

```
host, err := libp2p.New()

if err != nil {

return err

}

defer host.Close()

// 사용자 정의 부트스트랩 설정

cfg := bootstrap.BootstrapConfig{

MinPeerThreshold: 6, // 최소 피어 수

Period: 15 * time.Second, // 확인 주기

ConnectionTimeout: 10 * time.Second, // 연결 타임아웃

BackupBootstrapInterval: 30 * time.Minute, // 백업 저장 주기

MaxBackupBootstrapSize: 30, // 최대 백업 피어 수

BootstrapPeers: func() []peer.AddrInfo {

// 동적으로 부트스트랩 피어 반환

return getBootstrapPeers()

},

}

// 부트스트랩 시작

bootstrapper, err := bootstrap.Bootstrap(host.ID(), host, nil, cfg)

if err != nil {

return err

}

defer bootstrapper.Close()
```

```
return  nil

}


func  getBootstrapPeers() []peer.AddrInfo {

// 실제 구현에서는 설정 파일이나 원격 소스에서 피어 정보를 가져올 수 있음

return []peer.AddrInfo{

// 부트스트랩 피어들...

}

}
```

고급 활용

백업 피어 시스템 사용

```
package main


import (

"context"

"encoding/json"

"os"

"path/filepath"

"github.com/ipfs/boxo/bootstrap"

"github.com/libp2p/go-libp2p/core/peer"
```

```
)


// 백업 피어를 파일에 저장하고 로드하는 함수들
func saveBackupPeers(ctx context.Context, peers []peer.AddrInfo) {
backupFile := filepath.Join(os.TempDir(), "backup_peers.json")
data, err := json.Marshal(peers)
if err != nil {
log.Printf("백업 피어 직렬화 실패: %v", err)
return
}
if err := os.WriteFile(backupFile, data, 0644); err != nil {
log.Printf("백업 피어 저장 실패: %v", err)
}
}


func loadBackupPeers(ctx context.Context) []peer.AddrInfo {
backupFile := filepath.Join(os.TempDir(), "backup_peers.json")
data, err := os.ReadFile(backupFile)
if err != nil {
log.Printf("백업 피어 로드 실패: %v", err)
return nil
}
var peers []peer.AddrInfo
```

```
if err := json.Unmarshal(data, &peers); err != nil {
log.Printf("백업 피어 역직렬화 실패: %v", err)
return nil
}
return peers
}


func bootstrapWithBackup() error {
host, err := libp2p.New()
if err != nil {
return err
}
defer host.Close()
// 기본 부트스트랩 피어들
bootstrapPeers := []peer.AddrInfo{
// 부트스트랩 피어들...
}
// 백업 피어 기능과 함께 설정 생성
cfg := bootstrap.BootstrapConfigWithPeers(
bootstrapPeers,
bootstrap.WithBackupPeers(loadBackupPeers, saveBackupPeers),
)
// 부트스트랩 시작
```

```
bootstrapper, err := bootstrap.Bootstrap(host.ID(), host, nil, cfg)
if err != nil {
return err
}
defer bootstrapper.Close()
return nil
}
```

라우팅과 함께 부트스트랩

```
import (
"github.com/libp2p/go-libp2p-kad-dht"
"github.com/libp2p/go-libp2p/core/routing"
)

func bootstrapWithRouting() error {
host, err := libp2p.New()
if err != nil {
return err
}
defer host.Close()
// DHT 라우팅 생성
dht, err := dht.New(context.Background(), host)
```

```
if err != nil {
return err
}
// 부트스트랩 피어 설정
bootstrapPeers := []peer.AddrInfo{
// 부트스트랩 피어들...
}
cfg := bootstrap.BootstrapConfigWithPeers(bootstrapPeers)
// 라우팅과 함께 부트스트랩 시작
bootstrapper, err := bootstrap.Bootstrap(host.ID(), host, dht, cfg)
if err != nil {
return err
}
defer bootstrapper.Close()
return nil
}
```

설정 옵션

BootstrapConfig 필드

```
type BootstrapConfig struct {
// MinPeerThreshold: 더 많은 연결을 부트스트랩할지 결정하는 임계값
```

```
// 노드의 열린 연결 수가 이 수보다 적으면 부트스트랩 노드에 연결 시도

MinPeerThreshold  int

// Period: 노드가 주기적으로 부트스트랩을 시도하는 간격

// 부트스트랩 프로세스는 비용이 크지 않으므로 작은 값(<=30초) 사용 가능

Period time.Duration

// ConnectionTimeout: 부트스트랩 연결 시도를 취소하기 전 대기 시간

ConnectionTimeout time.Duration

// BootstrapPeers: 부트스트랩 프로세스에서 사용할 부트스트랩 피어 집합을 반환하는 함수

// 클라이언트가 언제든지 프로세스에서 사용하는 피어를 제어할 수 있게 함

BootstrapPeers  func() []peer.AddrInfo

// BackupBootstrapInterval: 연결된 노드를 임시 부트스트랩 피어로 저장하는 주기

BackupBootstrapInterval time.Duration

// MaxBackupBootstrapSize: 백업 부트스트랩 피어로 저장할 최대 피어 수

MaxBackupBootstrapSize  int

}
```

기본 설정값

```
var  DefaultBootstrapConfig = BootstrapConfig{

MinPeerThreshold: 4, // 최소 4개 피어 연결 유지

Period: 30 * time.Second, // 30초마다 확인

ConnectionTimeout: 10 * time.Second, // 10초 연결 타임아웃

BackupBootstrapInterval: 1 * time.Hour, // 1시간마다 백업 저장
```

```
MaxBackupBootstrapSize: 20, // 최대 20개 백업 피어

}
```

실제 예제

완전한 부트스트랩 시스템

```
package main


import (

"context"

"encoding/json"

"fmt"

"log"

"os"

"path/filepath"

"time"

"github.com/ipfs/boxo/bootstrap"

"github.com/libp2p/go-libp2p"

"github.com/libp2p/go-libp2p/core/peer"

"github.com/multiformats/go-multiaddr"

)
```

```go
type BootstrapManager struct {
host host.Host
bootstrapper io.Closer
configDir  string
}

func  NewBootstrapManager(configDir  string) (*BootstrapManager, error) {
// libp2p 호스트 생성
host, err := libp2p.New()
if  err != nil {
return  nil, fmt.Errorf("호스트 생성 실패: %w", err)
}
return &BootstrapManager{
host: host,
configDir: configDir,
}, nil
}

func (bm *BootstrapManager) Start() error {
// 부트스트랩 피어 로드
bootstrapPeers, err := bm.loadBootstrapPeers()
if  err != nil {
return  fmt.Errorf("부트스트랩 피어 로드 실패: %w", err)
```

```
}

// 부트스트랩 설정

cfg := bootstrap.BootstrapConfig{

MinPeerThreshold: 4,

Period: 30 * time.Second,

ConnectionTimeout: 10 * time.Second,

BackupBootstrapInterval: 1 * time.Hour,

MaxBackupBootstrapSize: 20,

BootstrapPeers: func() []peer.AddrInfo {

return  bootstrapPeers

},

}

// 백업 피어 기능 설정

cfg.SetBackupPeers(bm.loadBackupPeers, bm.saveBackupPeers)

// 부트스트랩 시작

bootstrapper, err := bootstrap.Bootstrap(bm.host.ID(), bm.host, nil, cfg)

if  err != nil {

return  fmt.Errorf("부트스트랩 시작 실패: %w", err)

}

bm.bootstrapper = bootstrapper

log.Printf("부트스트랩 시작됨 (피어 ID: %s)", bm.host.ID())

return  nil

}
```

```go
func (bm *BootstrapManager) Stop() error {
if  bm.bootstrapper != nil {
bm.bootstrapper.Close()
}
if  bm.host != nil {
return  bm.host.Close()
}
return  nil
}

func (bm *BootstrapManager) GetConnectedPeers() []peer.ID {
return  bm.host.Network().Peers()
}

func (bm *BootstrapManager) loadBootstrapPeers() ([]peer.AddrInfo, error) {
// 실제 구현에서는 설정 파일에서 로드
return []peer.AddrInfo{
{
ID: "12D3KooWGRPLCuizAiJWLtktrdxKKoKG5b3BtG4pEKbpzTLxmdSZ",
Addrs: []multiaddr.Multiaddr{
multiaddr.StringCast("/ip4/104.131.131.82/tcp/4001"),
},
```

```
},

// 더 많은 부트스트랩 피어들...

}, nil

}


func (bm *BootstrapManager) saveBackupPeers(ctx context.Context, peers []peer.Ad

backupFile := filepath.Join(bm.configDir, "backup_peers.json")

data, err := json.Marshal(peers)

if err != nil {

log.Printf("백업 피어 직렬화 실패: %v", err)

return

}

if err := os.WriteFile(backupFile, data, 0644); err != nil {

log.Printf("백업 피어 저장 실패: %v", err)

return

}

log.Printf("%d개 백업 피어 저장됨", len(peers))

}


func (bm *BootstrapManager) loadBackupPeers(ctx context.Context) []peer.AddrInfo

backupFile := filepath.Join(bm.configDir, "backup_peers.json")

data, err := os.ReadFile(backupFile)

if err != nil {
```

```
if !os.IsNotExist(err) {
log.Printf("백업 피어 로드 실패: %v", err)
}
return nil
}
var peers []peer.AddrInfo
if err := json.Unmarshal(data, &peers); err != nil {
log.Printf("백업 피어 역직렬화 실패: %v", err)
return nil
}
log.Printf("%d개 백업 피어 로드됨", len(peers))
return peers
}


func main() {
// 설정 디렉토리 생성
configDir := filepath.Join(os.TempDir(), "ipfs-bootstrap-example")
if err := os.MkdirAll(configDir, 0755); err != nil {
log.Fatal(err)
}
// 부트스트랩 매니저 생성 및 시작
manager, err := NewBootstrapManager(configDir)
if err != nil {
```

```
log.Fatal(err)

}

defer manager.Stop()

if err := manager.Start(); err != nil {

log.Fatal(err)

}

// 주기적으로 연결 상태 확인

ticker := time.NewTicker(10 * time.Second)

defer ticker.Stop()

for i := 0; i < 6; i++ {

<-ticker.C

peers := manager.GetConnectedPeers()

log.Printf("현재 연결된 피어 수: %d", len(peers))

if len(peers) > 0 {

log.Printf("연결된 피어들: %v", peers[:min(3, len(peers))])

}

}

}



func min(a, b int) int {

if a < b {

return a

}
```

```
return b

}
```

모범 사례

1. 적절한 임계값 설정

```
// 일반적인 IPFS 노드: 4-8개 피어
cfg.MinPeerThreshold = 6

// 서버 환경: 더 많은 연결 유지
cfg.MinPeerThreshold = 10

// 제한된 환경: 최소한의 연결
cfg.MinPeerThreshold = 2
```

2. 네트워크 조건에 맞는 타임아웃 설정

```
// 빠른 네트워크
cfg.ConnectionTimeout = 5 * time.Second
cfg.Period = 15 * time.Second

// 느린 네트워크
cfg.ConnectionTimeout = 30 * time.Second
```

```
cfg.Period = 60 * time.Second
```

3. 백업 피어 시스템 활용

```
// 백업 피어 기능 활성화
cfg.SetBackupPeers(loadBackupPeers, saveBackupPeers)

// 적절한 백업 크기 설정
cfg.MaxBackupBootstrapSize = 30  // 충분한 백업 피어 유지

// 적절한 백업 주기 설정
cfg.BackupBootstrapInterval = 30 * time.Minute  // 자주 업데이트
```

문제 해결

일반적인 문제들

1. 부트스트랩 피어 연결 실패

```
// 로그 레벨 설정으로 디버깅
import  "github.com/ipfs/go-log/v2"

logging.SetLogLevel("bootstrap", "DEBUG")
```

```
// 또는 환경 변수로
// GOLOG_LOG_LEVEL=bootstrap=DEBUG
```

2. 네트워크 연결 문제

```
// 연결 타임아웃 증가
cfg.ConnectionTimeout = 60 * time.Second

// 더 자주 재시도
cfg.Period = 10 * time.Second
```

3. 부트스트랩 피어 부족

```
// 더 많은 부트스트랩 피어 추가
bootstrapPeers := append(bootstrapPeers, additionalPeers...)

// 백업 피어 시스템 활용
cfg.SetBackupPeers(loadBackupPeers, saveBackupPeers)
```

모니터링

```
func monitorBootstrap(host host.Host) {
ticker := time.NewTicker(30 * time.Second)
defer ticker.Stop()
```

```
for {
select {
case <-ticker.C:
peers := host.Network().Peers()
log.Printf("연결된 피어 수: %d", len(peers))
if  len(peers) < 4 {
log.Warn("연결된 피어 수가 부족합니다")
}
}
}
}
```

이 문서는 Bootstrap 모듈의 핵심 기능과 사용법을 다룹니다. 더 자세한 정보는 Autoconf 자동 설정과 Peering 피어링 문서를 참조하세요.